Практическая 2. Отчет
Ответ 1. В функции send_reliable создаётся сокет с типом SOCK_STREAM и вызывается connect() — это протокол TCP. SOCK_STREAM означает потоковый сокет, который обеспечивает надёжную доставку данных с установлением соединения, а connect() как раз и устанавливает это соединение с сервером.

Ответ 2. В функции send_fast создаётся сокет с типом SOCK_DGRAM и вызывается sendto() — это протокол UDP. SOCK_DGRAM означает датаграммный сокет, который работает без установления соединения, а sendto() отправляет отдельную датаграмму по указанному адресу.

Ответ 3. Данные от клиента к серверу передаются в формате JSON. Это подтверждает строка в client.py:

python
client.send(json.dumps(message, ensure_ascii=False).encode("utf-8"))
Здесь json.dumps() превращает объект Python в JSON-строку, которая затем кодируется в байты UTF-8 и отправляется.

Ответ 4. Если клиент отправит обычную строку "hello", сервер попытается разобрать её через json.loads(), что вызовет исключение json.JSONDecodeError. Сервер выведет в консоль сообщение [TCP] Ошибка: не удалось разобрать JSON от ... и отправит клиенту ответ {"status": "ERROR", "reason": "invalid JSON"}.

2) Тестирование граничных случаев
Случай 1. Сервер выключен, клиент TCP

При запуске клиента с выключенным сервером возникает ошибка ConnectionRefusedError: [WinError 10061] No connection could be made because the target machine actively refused it. Это происходит потому, что TCP требует установления соединения через connect(), а сервер не слушает порт — операционная система отклоняет подключение. Программа завершается с необработанным исключением, так как в коде нет обработки этой ошибки.

Случай 2. Сервер выключен, клиент UDP

После комментирования вызова TCP-клиента и запуска только UDP-части: клиент отправляет датаграмму через sendto() и ждёт ответ с таймаутом 2 секунды. Ответ не приходит, срабатывает socket.timeout, и клиент возвращает {"status": "TIMEOUT"}. Клиент ждал ровно 2 секунды (заданный таймаут). Это происходит потому, что UDP не требует установления соединения — отправка проходит «молча», но ответить некому.

Случай 3. Клиент отправляет не JSON

После замены строки на client.send(b"hello") сервер получил байты hello, попытался разобрать их через json.loads() и получил json.JSONDecodeError. В консоли сервера появилось: [TCP] Ошибка: не удалось разобрать JSON от ('127.0.0.1', ...). Клиенту сервер отправил ответ {"status": "ERROR", "reason": "invalid JSON"}. Программа корректно обработала некорректные данные благодаря блоку try/except.

Случай 4. Сравнение форматов

Вывод format_check.py:

text
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 72 байт
XML: <weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather>
Размер XML: 113 байт
JSON компактнее (72 байта против 113). Это объясняется тем, что XML требует закрывающих тегов для каждого элемента и более многословен, тогда как JSON использует короткий синтаксис «ключ: значение» с разделителями. При этом оба формата позволяют восстановить исходные данные.

3) Вывод
Для данного приложения я бы выбрал TCP + JSON. TCP обеспечивает надёжную доставку данных и гарантирует, что сообщение дойдёт до сервера в правильном порядке — это важно, так как потеря данных

о погоде или её искажение недопустимы. JSON компактнее XML, проще читается и удобно преобразуется в объекты Python через стандартную библиотеку json. UDP имел бы смысл только при потоковой передаче с высокой частотой, где потеря отдельного пакета не критична, а JSON предпочтительнее XML из-за меньшего размера и простоты парсинга.